

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

09 - Árvores Balanceadas

Prof. Alex Kutzke

Junho de 2021

Tradução adaptada dos slides presentes em [1]. Imagens e exemplos também retirados do mesmo material.

[1] <<https://algs4.cs.princeton.edu/lectures/keynote/23Quicksort.pdf>>

Roteiro

Árvores 2-3

Árvores Red-Black

Inserção em Árvores Red-Black

Árvores 2-3

Revisão de Implementações de Tabela de Símbolos

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2} N$	N	$\frac{1}{2} N$		<code>equals()</code>
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2} N$	$\frac{1}{2} N$	✓	<code>compareTo()</code>
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	<code>compareTo()</code>
goal	$\log N$	$\log N$	$\log N$	$\log N$	$\log N$	$\log N$	✓	<code>compareTo()</code>

Fonte: (SEdgeWICK; WAYNE, 2014)

Desafio garantia de performance.

Árvores 2-3

Permite **uma** ou **duas** chaves por nó:

- Nó-2: **uma chave**, **dois filhos**;
- Nó-3: **duas chaves**, **três filhos**;

Propriedades:

Ordem simétrica percurso in-ordem percorre as chaves em ordem crescente;

Balanceamento perfeito Todos os caminhos da raiz até as folhas tem o mesmo comprimento;

Árvores 2-3

Permite **uma** ou **duas** chaves por nó:

- Nó-2: **uma chave**, **dois filhos**;
- Nó-3: **duas chaves**, **três filhos**;

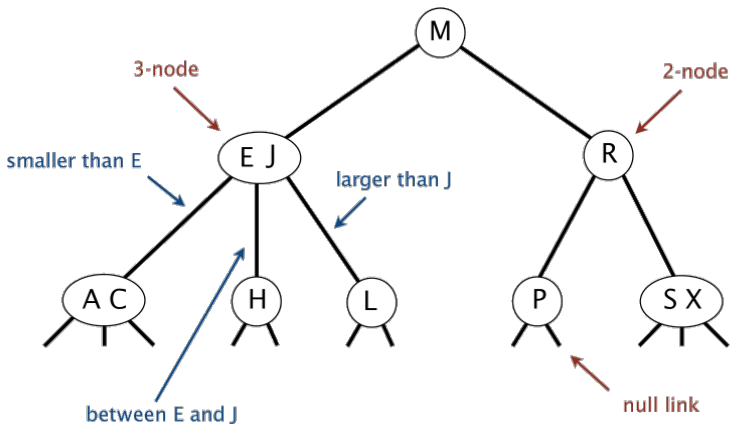
Propriedades:

Ordem simétrica percurso in-ordem percorre as chaves em ordem crescente;

Balanceamento perfeito Todos os caminhos da raiz até as folhas tem o mesmo comprimento;

Como manter o balanceamento perfeito?

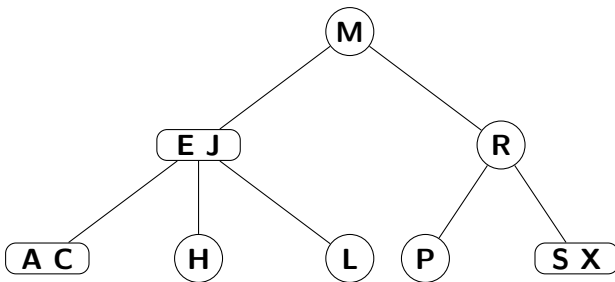
Exemplo: árvore 2-3



Fonte: (SEdgeWICK; WAYNE, 2014)

Busca

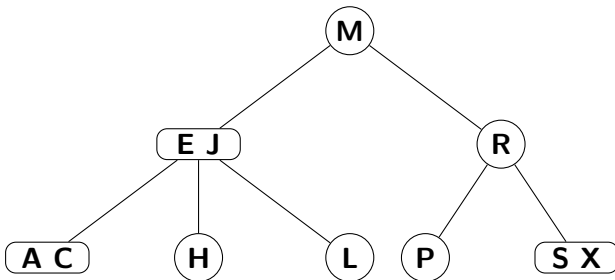
1. Compara o elemento buscado com as chaves da raiz;
2. Encontra o intervalo que contém o elemento busca;
3. Busca recursivamente através do filho associado ao intervalo;



Busca

1. Compara o elemento buscado com as chaves da raiz;
2. Encontra o intervalo que contém o elemento busca;
3. Busca recursivamente através do filho associado ao intervalo;

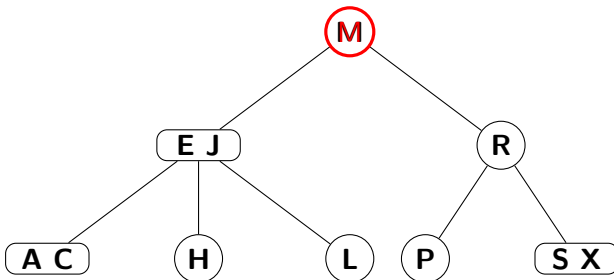
Busca por **H**



Busca

1. Compara o elemento buscado com as chaves da raiz;
2. Encontra o intervalo que contém o elemento busca;
3. Busca recursivamente através do filho associado ao intervalo;

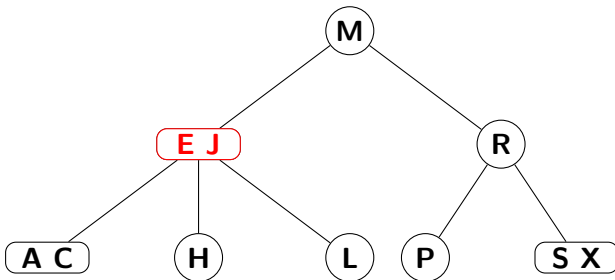
Busca por **H**



Busca

1. Compara o elemento buscado com as chaves da raiz;
2. Encontra o intervalo que contém o elemento busca;
3. Busca recursivamente através do filho associado ao intervalo;

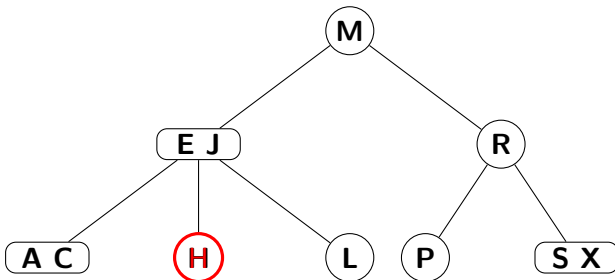
Busca por **H**



Busca

1. Compara o elemento buscado com as chaves da raiz;
2. Encontra o intervalo que contém o elemento busca;
3. Busca recursivamente através do filho associado ao intervalo;

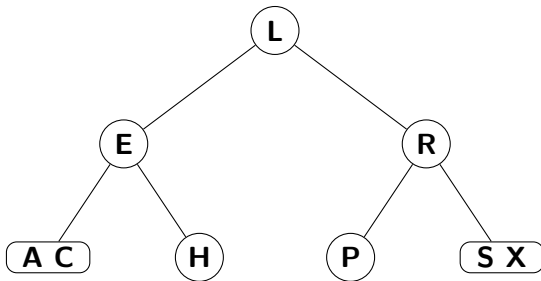
Busca por **H**



Inserção

Inserção em um nó-2 folha

1. Adiciona chave a nó-2, transformando-o em um nó-3;

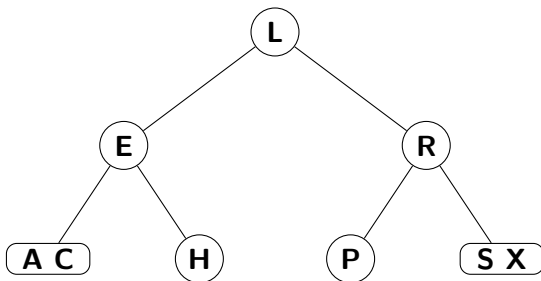


Inserção

Inserção em um nó-2 folha

1. Adiciona chave a nó-2, transformando-o em um nó-3;

Inserir **G**

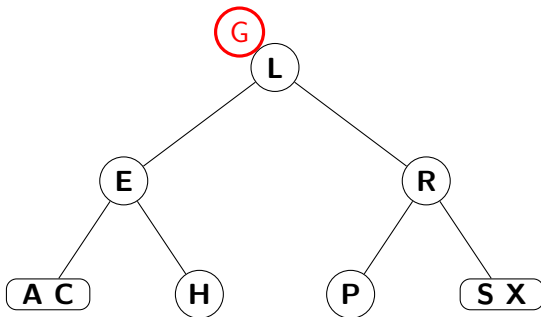


Inserção

Inserção em um nó-2 folha

1. Adiciona chave a nó-2, transformando-o em um nó-3;

Inserir **G**

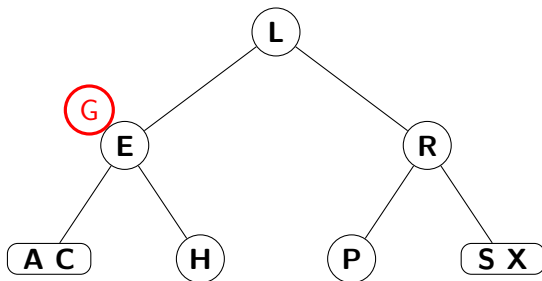


Inserção

Inserção em um nó-2 folha

1. Adiciona chave a nó-2, transformando-o em um nó-3;

Inserir **G**

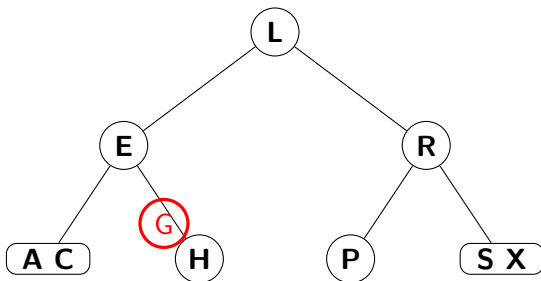


Inserção

Inserção em um nó-2 folha

1. Adiciona chave a nó-2, transformando-o em um nó-3;

Inserir **G**

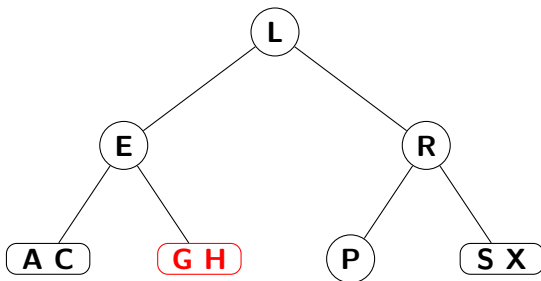


Inserção

Inserção em um nó-2 folha

1. Adiciona chave a nó-2, transformando-o em um nó-3;

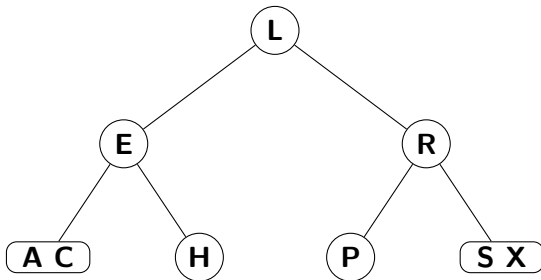
Inserir **G**



Inserção

Inserção em um nó-3 folha

1. Adiciona chave a nó-3, transformando-o em, temporariamente, um nó-4;
2. Mova a chave intermediária do nó-4 para o nó pai;
3. Repita o processo na árvore acima se necessário;
4. Se alcançar a raiz e esta se tornar um nó-4, dividir em 3 nós-2;

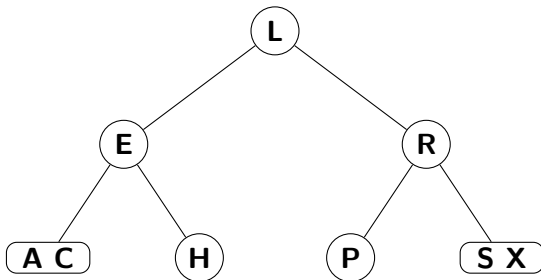


Inserção

Inserção em um nó-3 folha

1. Adiciona chave a nó-3, transformando-o em, temporariamente, um nó-4;
2. Mova a chave intermediária do nó-4 para o nó pai;
3. Repita o processo na árvore acima se necessário;
4. Se alcançar a raiz e esta se tornar um nó-4, dividir em 3 nós-2;

Inserir **Z**

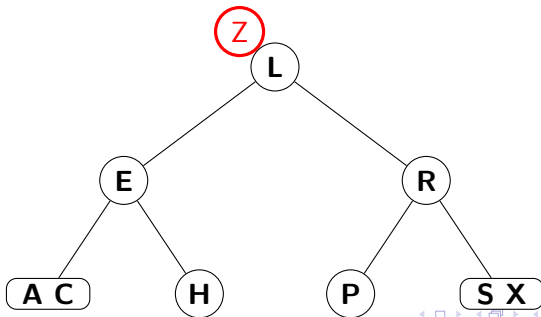


Inserção

Inserção em um nó-3 folha

1. Adiciona chave a nó-3, transformando-o em, temporariamente, um nó-4;
2. Mova a chave intermediária do nó-4 para o nó pai;
3. Repita o processo na árvore acima se necessário;
4. Se alcançar a raiz e esta se tornar um nó-4, dividir em 3 nós-2;

Inserir **Z**

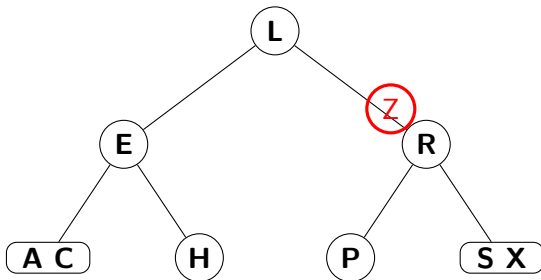


Inserção

Inserção em um nó-3 folha

1. Adiciona chave a nó-3, transformando-o em, temporariamente, um nó-4;
2. Mova a chave intermediária do nó-4 para o nó pai;
3. Repita o processo na árvore acima se necessário;
4. Se alcançar a raiz e esta se tornar um nó-4, dividir em 3 nós-2;

Inserir **Z**

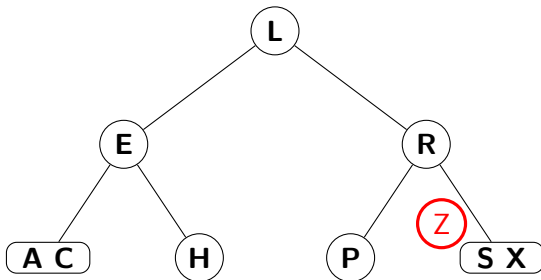


Inserção

Inserção em um nó-3 folha

1. Adiciona chave a nó-3, transformando-o em, temporariamente, um nó-4;
2. Mova a chave intermediária do nó-4 para o nó pai;
3. Repita o processo na árvore acima se necessário;
4. Se alcançar a raiz e esta se tornar um nó-4, dividir em 3 nós-2;

Inserir **Z**

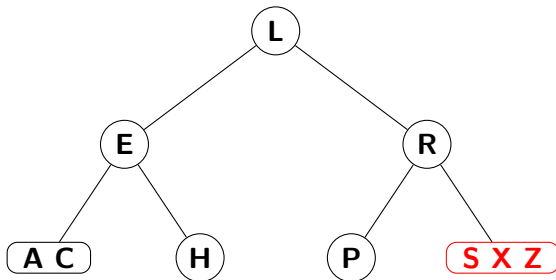


Inserção

Inserção em um nó-3 folha

1. Adiciona chave a nó-3, transformando-o em, temporariamente, um nó-4;
2. Mova a chave intermediária do nó-4 para o nó pai;
3. Repita o processo na árvore acima se necessário;
4. Se alcançar a raiz e esta se tornar um nó-4, dividir em 3 nós-2;

Inserir **Z**

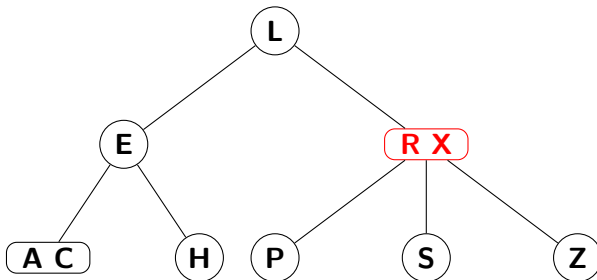


Inserção

Inserção em um nó-3 folha

1. Adiciona chave a nó-3, transformando-o em, temporariamente, um nó-4;
2. Mova a chave intermediária do nó-4 para o nó pai;
3. Repita o processo na árvore acima se necessário;
4. Se alcançar a raiz e esta se tornar um nó-4, dividir em 3 nós-2;

Inserir **Z**



Exemplo: inserção

Inserir S

Exemplo: inserção

Inserir X



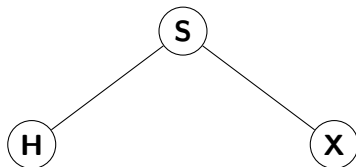
Exemplo: inserção

Insérer **H**

S X

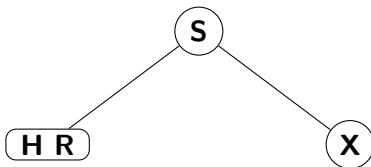
Exemplo: inserção

Inserir **R**



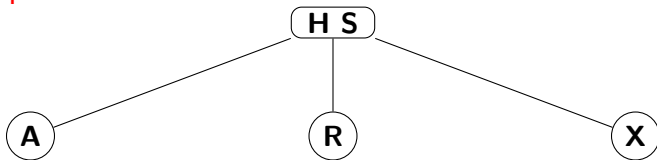
Exemplo: inserção

Inserir **A**



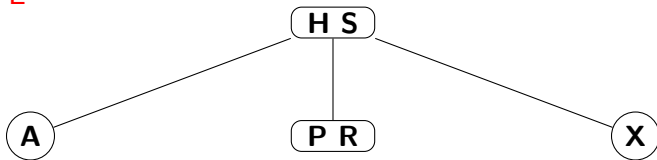
Exemplo: inserção

Inserir **P**



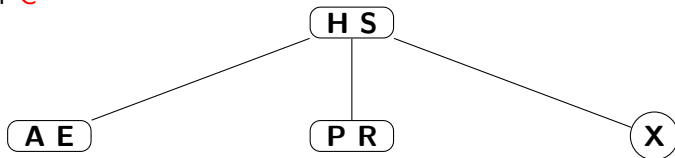
Exemplo: inserção

Inserir **E**



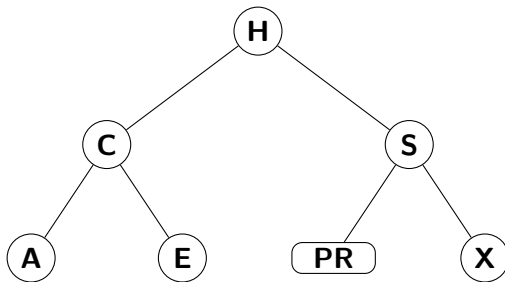
Exemplo: inserção

Inserir **C**



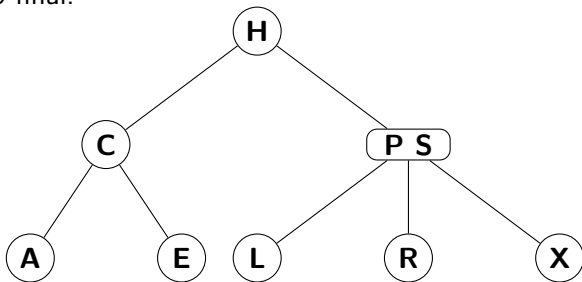
Exemplo: inserção

Inserir **L**



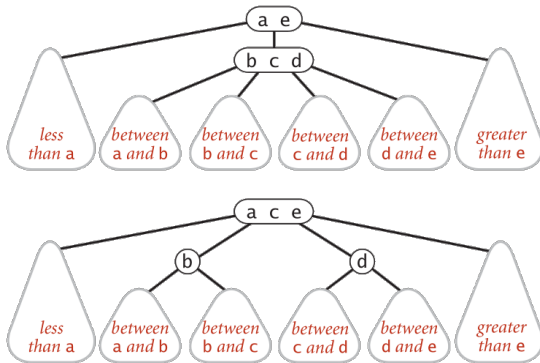
Exemplo: inserção

Resultado final:



Transformações locais

Dividir um **nó-4** temporário em um **nó-3** é uma **operação local**:
número de operações necessárias é constante.



Fonte: (SEdgeWICK; WAYNE, 2014)

Propriedades

Invariantes: Mantém ordem simétrica e balanceamento perfeito;
Por que? Cada transformação mantém ordem simétrica e balanceamento perfeito;

root



parent is a 3-node

left



parent is a 2-node

left



middle



right



right



Fonte: (SEdgeWICK; WAYNE, 2014)

Sumário de Implementações de Tabela de Símbolos

[illegible]

Fonte: (SEDGEWICK; WAYNE, 2014)

Constante **c** depende da implementação.

Árvores Red-Black

Como implementar Árvores 2-3 com Árvores Binárias?

Desafio Como representar um nó-3?

Abordagem 1: árvore binária simples

- Sem forma de diferenciar um nó-3 de um nó-2;
- Impossível mapear uma árvore binária de volta e uma 2-3;



Abordagem 2: árvore binária com nós “cola”

- Desperdiça espaço, desperdiça links;
- Código, provavelmente, confuso;



Abordagem 3: árvore binária com arestas “cola” vermelhas

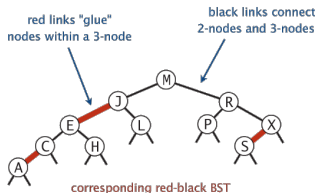
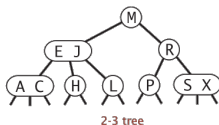
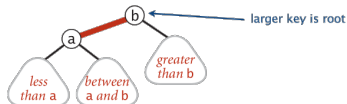
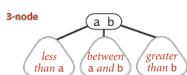
- Bastante utilizado na prática;
- Restrição abstrata: arestas vermelhas descem para esquerda;



Árvores Left-leaning Red-black

(Guibas-Sedgwick 1979 and Sedgwick 2007)

1. Representa árvores 2-3 com árvores binárias;
2. Usa **arestas descendentes à esquerda** como “cola” para nós-3.

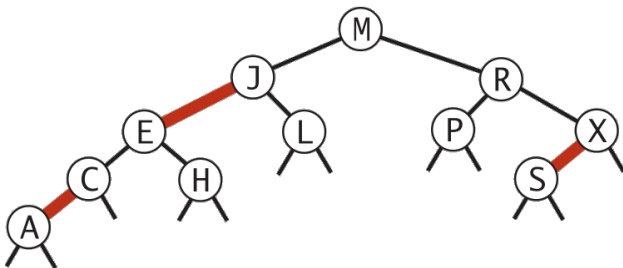


Fonte: (SEDGWICK; WAYNE, 2014)

Árvores Left-leaning Red-black: definição

Uma árvore binária tal que:

- Nenhum nó tem duas arestas vermelhas conectadas a si;
- Todos os caminhos da raiz até as folhas possuem o mesmo número de arestas pretas “balanceamento perfeito preto”;
- Arestas vermelhas sempre são descendentes à esquerda.

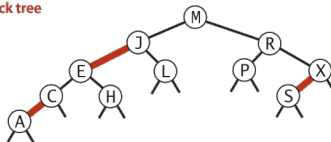


Fonte: (SEdgeWICK; WAYNE, 2014)

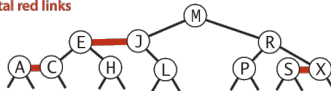
Correspondência direta com Árvores 2-3

Propriedade chave Possui correspondência direta com árvores 2-3:

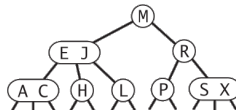
red-black tree



horizontal red links



2-3 tree

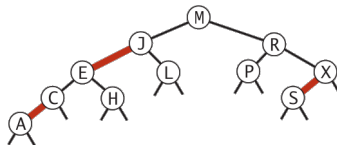


Fonte: (SEdgeWICK; WAYNE, 2014)

Busca

Operação de busca é a mesma de Árvores Binárias Simples (basta ignorar a cor): Mas é mais rápido pois a árvore red-black tem um balanceamento melhor

```
public Val get(Key key)
{
    Node x = root;
    while (x != null)
    {
        int cmp = key.compareTo(x.key);
        if (cmp < 0) x = x.left;
        else if (cmp > 0) x = x.right;
        else if (cmp == 0) return x.val;
    }
    return null;
}
```



Fonte: (SEdgeWICK; WAYNE, 2014)

Observação Maior parte das outras operações também são idênticas.

Representação

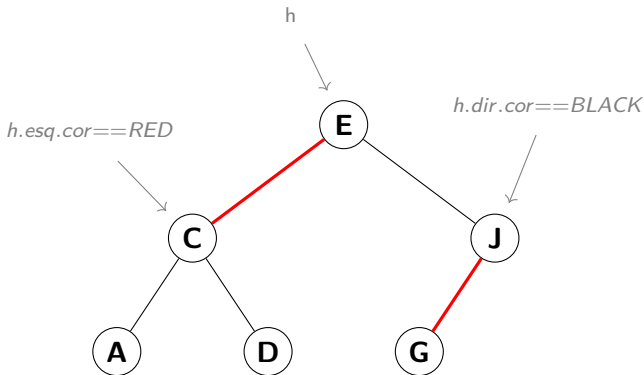
Informação da cor pode ser armazenada no nó filho da aresta.

```

1  #define RED    1
2  #define BLACK  0
3
4  typedef struct arvoreRB {
5      int info;
6      int cor;
7      struct arvoreRB *esq;
8      struct arvoreRB *dir;
9  } ArvoreRB;
10
11 int eh_no_vermelho(ArvoreRB * no){
12     if(!no) return BLACK;
13     return(no->cor == RED);
14 }

```

Representação: exemplo

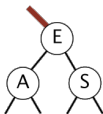


Inserção Árvore RB

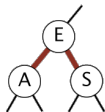
Estratégia básica: Manter correspondência direta com árvores 2-3.

Durante operações internas, manter:

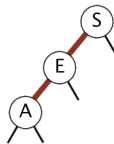
- Ordem simétrica;
- Balanceamento perfeito de arestas pretas;
mas não necessariamente manter cores.



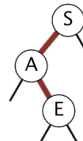
right-leaning
red link



two red children
(a temporary 4-node)



left-left red
(a temporary 4-node)



left-right red
(a temporary 4-node)

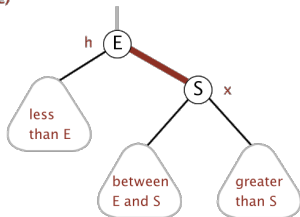
Fonte: (SEdGEWICK; WAYNE, 2014)

Como? Aplicar operações elementares de **rotação** e **troca de cor**.

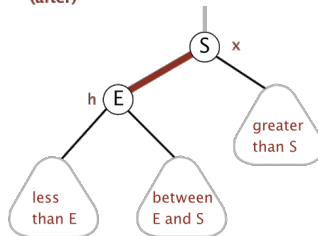
Operações elementares de Árvores RB

Rotação à esquerda Manter correspondência direta com árvores 2-3.

rotate E left
(before)



rotate E left
(after)



Fonte: (SEdgeWICK; WAYNE, 2014)

Fonte: (SEdgeWICK; WAYNE, 2014)

Invariantes: Manter ordem simétrica e balanceamento perfeito de arestas pretas.

Operações elementares de Árvores RB

Rotação à esquerda Manter correspondência direta com árvores 2-3.

```

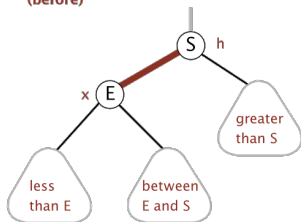
1  ArvoreRB * rot_esq(ArvoreRB * no){
2      ArvoreRB * x = no->dir;
3      no->dir = x->esq;
4      x->esq = no;
5      x->cor = no->cor;
6      no->cor = RED;
7      return(x);
8  }
```

Invariantes: Manter ordem simétrica e balanceamento perfeito de arestas pretas.

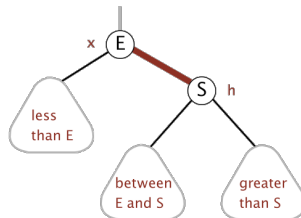
Operações elementares de Árvores RB

Rotação à direita Passar uma aresta vermelha temporariamente à direita.

rotate S right
(before)



rotate S right
(after)



Fonte: (SEdgeWICK; WAYNE, 2014)

Fonte: (SEdgeWICK; WAYNE, 2014)

Invariantes: Manter ordem simétrica e balanceamento perfeito de arestas pretas.

Operações elementares de Árvores RB

Rotação à direita Passar uma aresta vermelha temporariamente à direita.

```

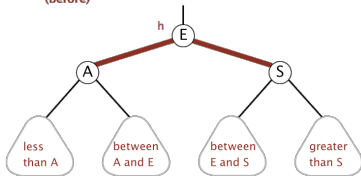
1  ArvoreRB * rot_dir(ArvoreRB * no){
2      ArvoreRB * x = no->esq;
3      no->left = x->dir;
4      x->dir = no;
5      x->cor = no->cor;
6      no->cor = RED;
7      return(x);
8  }
```

Invariantes: Manter ordem simétrica e balanceamento perfeito de arestas pretas.

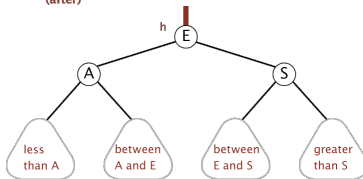
Operações elementares de Árvores RB

Color flip Recolorir um nó-4 temporário para dividi-lo.

flip colors
(before)



flip colors
(after)



Fonte: (SEdgeWICK; WAYNE, 2014)

Fonte: (SEdgeWICK; WAYNE, 2014)

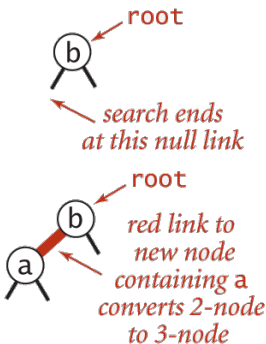
Invariantes: Manter ordem simétrica e balanceamento perfeito de arestas pretas.

Inserção em Árvores Red-Black

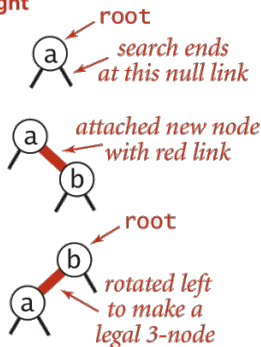
Inserção Árvores RB

Aquecimento 1: Inserir em uma árvore com um único no.

left



right



Fonte: (SEDGEWICK; WAYNE, 2014)

Inserção Árvores RB

Caso 1: Inserir em um nó-2 folha.

- Inserção normal de árvore binária; nova aresta é vermelha; para manter ordem simétrica e balanceamento perfeito de arestas pretas
- Se a nova aresta vermelha descende à direita, rotaciona para esquerda. para corrigir o invariante das cores

insert C



right link red
so rotate left

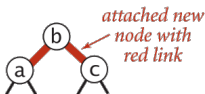


Fonte: (SEGEWICK; WAYNE, 2014)

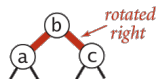
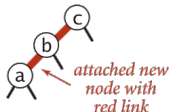
Inserção Árvores RB

Aquecimento 2: Inserir em uma árvore com exatamente 2 nós.

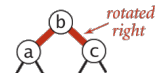
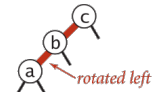
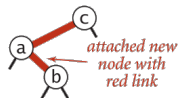
larger



smaller



between

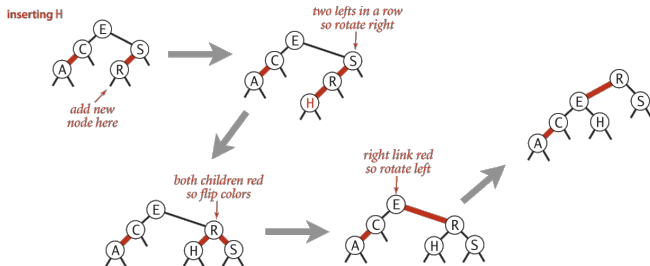


Fonte: (SEDGEWICK; WAYNE, 2014)

Inserção Árvores RB

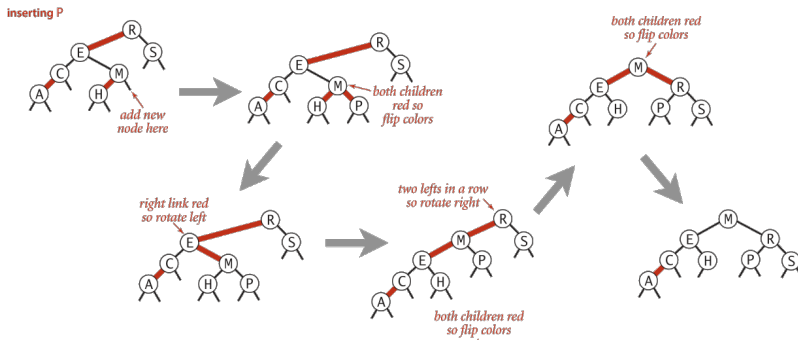
Caso 2: Inserir em um nó-3 folha.

- Inserção normal de árvore binária; nova aresta é vermelha; para manter ordem simétrica e balanceamento perfeito de arestas pretas
- Rotaciona para balancear o nó-4 temporário (se necessário);
- Trocar cores para passar a aresta vermelha para o nível acima;
- Rotacionar para aresta vermelha descender para esquerda (se necessário); para corrigir o invariante das cores



Inserção Árvores RB

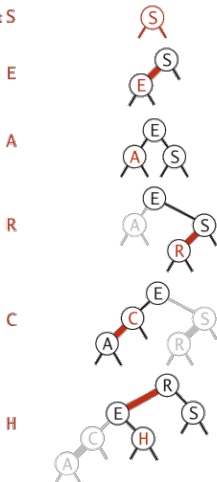
Finalização: Se necessário, repita casos 1 e 2 nos níveis anteriores da árvore.



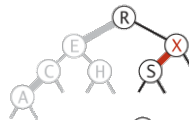
Fonte: (SEdgeWICK; WAYNE, 2014)

Inserção Árvore RB - Exemplo

insert S



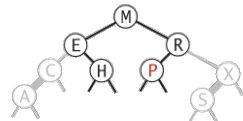
X



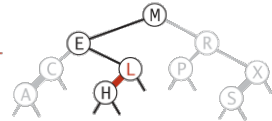
M



P



L



Fonte: (SEdgeWICK; WAYNE, 2014)

Inserção Árvore RB - Exemplo ordem crescente

insert A

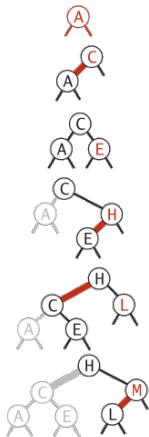
C

E

H

L

M

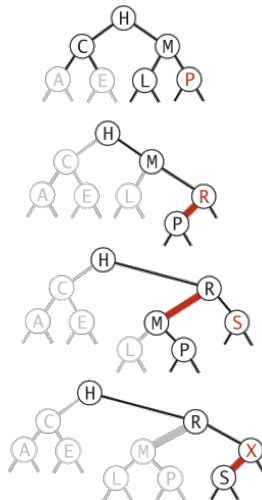


P

R

S

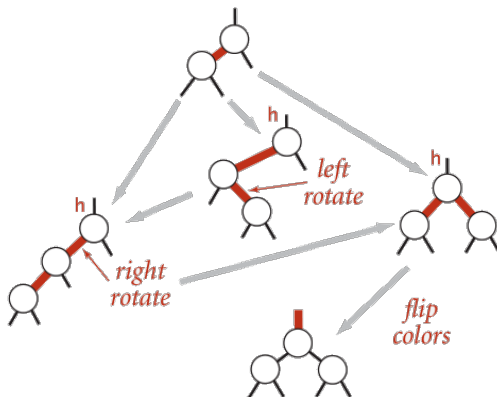
X



Inserção: implementação

Mesmo código para todos os casos

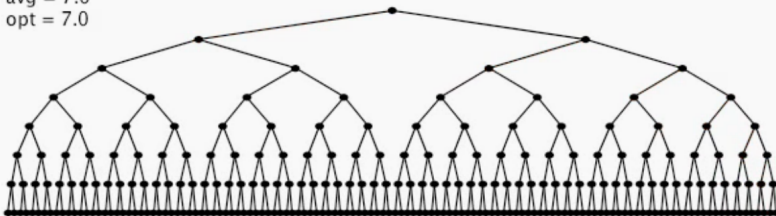
- Filho à direita vermelho, filho à esquerda preto: **rotaciona à esquerda**;
- Filho à esquerda vermelho, neto à esquerda vermelho: **rotaciona à direita**;
- Ambos filhos vermelhos: **troca cores**;



Fonte: (SEGEWICK; WAYNE, 2014)

Exemplos

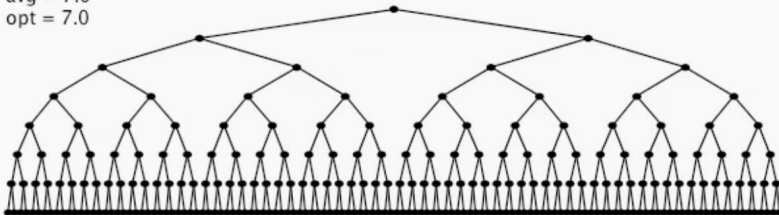
$N = 255$
 $\max = 8$
 $\text{avg} = 7.0$
 $\text{opt} = 7.0$



255 inserções em ordem crescente.
Fonte: (SEdgeWICK; WAYNE, 2014)

Exemplos

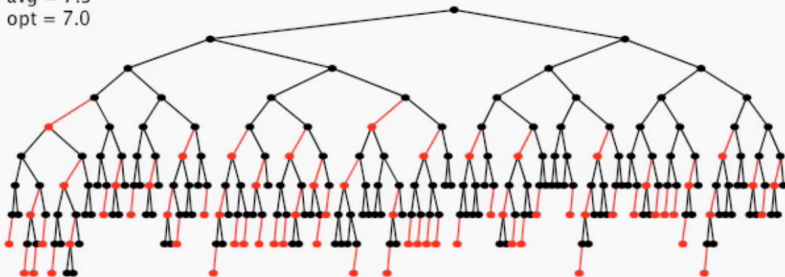
$N = 255$
 $\max = 8$
 $\text{avg} = 7.0$
 $\text{opt} = 7.0$



255 inserções em ordem decrescente.
Fonte: (SEdgeWICK; WAYNE, 2014)

Exemplos

N = 255
max = 10
avg = 7.3
opt = 7.0

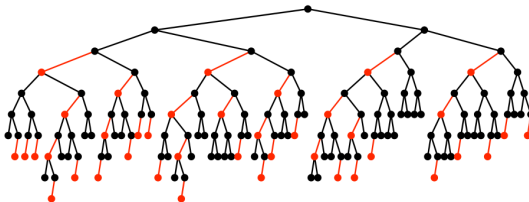


255 inserções em ordem aleatória.
Fonte: (SEdGEWICK; WAYNE, 2014)

Desempenho

Proposição Altura da árvore é $\leq 2 \log_2 N$ no pior caso.

- Todos os caminhos da raiz até as folhas tem o melhor número de arestas pretas;
- Nunca existem duas arestas vermelhas em sequência.



Fonte: (SEdgeWICK; WAYNE, 2014)

Propriedade Altura da árvore é $\sim 1.0 \log_2 N$ em aplicações típicas.

Resumo Implementações Tabela de Símbolos

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2} N$	N	$\frac{1}{2} N$		equals()
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2} N$	$\frac{1}{2} N$	✓	compareTo()
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	compareTo()
2-3 tree	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	✓	compareTo()
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$	$1.0 \lg N^*$	$1.0 \lg N^*$	$1.0 \lg N^*$	✓	compareTo()

Fonte: (SEdgeWICK; WAYNE, 2014)

* valor exato do coeficiente não é conhecido, mas é extremamente próximo de 1

Referências utilizadas na apresentação I

 SEDGEWICK, R.; WAYNE, K. *Algorithms: Part I*. [S.I.]: Addison-Wesley Professional, 2014.